BAGIAN 01

SMP Nurul Huda

Teknologi, alasan pemilihan, dan cara menjelaskan project

Panduan ini disusun untuk membantu anggota tim menjelaskan project website sekolah dan layanan pendaftaran siswa kepada dosen pembimbing. Gunakan sebagai bahan memahami sistem, bukan naskah yang harus dihafalkan persis.

Tanggal pemeriksaan source: 28 September 2026. Lingkupnya project lokal `project-k`. Dokumen ini tidak menyatakan sistem sudah dipakai sekolah atau sudah berjalan di VPS.

Inti penjelasan: kami membuat website informasi sekolah dan administrasi pendaftaran siswa. Tampilan menggunakan React dan TypeScript; fondasi API menggunakan Fastify; penyimpanan server dirancang menggunakan PostgreSQL dan Drizzle ORM. Saat ini sebagian besar fitur masih simulasi lokal.

Cara membaca

Kebutuhan saat bimbingan	Halaman
Pembukaan dan tujuan project	2
Teknologi dan alasan pemilihannya	3-6
Alur sistem, folder, dan progres nyata	7-8
Keamanan dan metode Agile	9-10
Pengujian serta rencana VPS	11
Latihan pertanyaan dosen	12-13
Persiapan demo dan pekerjaan berikutnya	14
Bukti source dan referensi resmi	15

Tiga label yang harus dibedakan

- **Ada di source:** kode atau konfigurasi sudah tersedia; belum tentu telah diuji dalam kondisi production.
- **Simulasi:** perilaku dapat dicoba di frontend, tetapi belum menggunakan penyimpanan server untuk fitur tersebut.
- **Rencana:** pekerjaan atau metode yang diusulkan; jangan disebut sudah dilaksanakan tanpa bukti.

Alasan pemilihan dalam dokumen ini merupakan pertimbangan teknis berdasarkan kebutuhan dan kode saat ini, bukan hasil eksperimen perbandingan teknologi. Contoh jawaban perlu disesuaikan dengan kontribusi dan kegiatan tim yang benar-benar dilakukan.

BAGIAN 02

Cerita project dalam satu menit

Contoh pembukaan

“Project kami adalah implementasi website informasi sekolah dan layanan pendaftaran siswa SMP Nurul Huda. Website publik menampilkan profil sekolah, program, struktur organisasi, guru, berita, dan formulir pendaftaran. Dashboard admin disiapkan untuk mengelola konten dan data pendaftar, termasuk input pendaftaran offline.

Kami menggunakan TypeScript pada frontend dan backend agar bentuk data lebih jelas dan bahasa yang dipelajari tim lebih konsisten. React dipakai untuk antarmuka, Fastify untuk API, dan PostgreSQL sebagai database yang direncanakan. Saat ini tampilan interaktif dan fondasi backend sudah tersedia, tetapi integrasi seluruh fitur dengan database masih menjadi pekerjaan berikutnya.

Pengerjaan direncanakan bertahap menggunakan prinsip Agile, dengan prioritas fitur, demo berkala, pengujian, dan perbaikan dari masukan sekolah.”

Masalah yang ingin ditangani

Informasi sekolah, publikasi kegiatan, serta pencatatan pendaftar diharapkan dapat dikelola dalam satu sistem. **Ini tujuan desain, bukan temuan observasi yang sudah terbukti.** Wawancara sekolah tetap diperlukan untuk memastikan proses saat ini, kendala pencarian data, dan kebutuhan dokumen.

Siapa penggunanya?

- Calon siswa/orang tua: mencari informasi sekolah dan mengisi pendaftaran.
- Admin pendaftaran: mencatat pendaftar online/offline dan memeriksa kelengkapan data.
- Pengelola konten: memperbarui berita, agenda, profil guru, dan struktur sekolah.
- Penanggung jawab sekolah: mengevaluasi informasi dan ringkasan layanan sesuai hak akses yang nanti ditetapkan.

Usulan judul untuk dibahas

Implementasi Sistem Informasi Sekolah dan Pendaftaran Siswa Berbasis Web Menggunakan React dan Framework Fastify dengan Metode Agile di SMP Nurul Huda

React secara teknis merupakan library UI, sedangkan Fastify merupakan framework backend. Penulisan “React dan Framework Fastify” menjaga perbedaan tersebut. Judul masih usulan dan perlu persetujuan dosen serta kesesuaian dengan ruang lingkup PKM.

Batas lingkup: satu sekolah; belum mencakup pembayaran, rapor, absensi, LMS, atau integrasi Dapodik. Jangan menjanjikan fitur tersebut jika belum disepakati.

BAGIAN 03

Peta teknologi yang benar-benar dipakai

Daftar berikut bersumber dari manifest package dan konfigurasi lokal. Angka menunjukkan versi utama yang dideklarasikan, bukan klaim versi paling baru. Versi terpasang yang tepat mengikuti `package-lock.json`.

Lapisan	Teknologi	Fungsi dalam project
Bahasa utama	TypeScript 5.9	Kode UI dan backend dengan pemeriksaan tipe.
Antarmuka	React 19	Komponen halaman, form, tabel, dan dashboard.
Tooling frontend	Vinext 1 beta + Vite 8	Routing/rendering bergaya Next.js dan proses dev/build.
Tampilan	CSS + Tailwind CSS 4	Layout, warna, jarak, responsivitas; banyak styling khusus ada di file CSS.
Ikon	Lucide React	Ikon antarmuka yang konsisten.
Runtime backend	Node.js	Menjalankan aplikasi backend; project meminta Node >=22.13.0.
Framework backend	Fastify 5	Routing HTTP, plugin, validasi request, dan respons API.
Database target	PostgreSQL 17 di Compose	Menyimpan data relasional di server.
Akses database	Drizzle ORM + Drizzle Kit	Schema TypeScript, query, dan migration SQL.
Validasi konfigurasi	Zod 4	Memeriksa environment saat backend dijalankan.
Kontrak integrasi	OpenAPI, format YAML	Acuan endpoint, field, respons, dan error.
Pemeriksaan kode	ESLint, tsc, Node test, Vitest	Lint, tipe, pengujian frontend, dan pengujian backend.
Lingkungan lokal/deploy	npm workspaces, Docker/Compose	Mengelola dua package; template layanan backend/database.
Tooling preview	Cloudflare Vite plugin, Wrangler	Bagian konfigurasi frontend lokal saat ini; bukan bukti sudah deploy ke Cloudflare.

Jawaban paling singkat: “Bahasa utamanya TypeScript. React untuk tampilan, Fastify di Node.js untuk backend, PostgreSQL untuk database, dan Drizzle untuk akses database.” Tambahkan detail tooling jika ditanya lebih lanjut.

Jangan tertukar: SQL adalah bahasa query; PostgreSQL adalah sistem database; Node.js adalah runtime; Agile adalah pendekatan pengerjaan. Semuanya bukan nama bahasa pemrograman utama yang berbeda-beda.

BAGIAN 04

Mengapa TypeScript dan React?

TypeScript: bentuk data lebih jelas

TypeScript memeriksa tipe sebelum program dijalankan. [1] Dalam project tim, tipe membantu menunjukkan field yang diharapkan komponen atau fungsi, misalnya nama siswa, nomor pendaftaran, dan status verifikasi. Alasan praktisnya adalah mengurangi salah penggunaan data ketika modul dikerjakan anggota berbeda.

Contoh: NISN sebaiknya diperlakukan sebagai teks agar angka nol di depan tetap terjaga. Pemeriksaan tipe membantu jika kode mencoba menggunakan angka ketika fungsi mengharapkan teks. Namun format dan panjang NISN tetap harus divalidasi saat menerima input.

Keterbatasan: TypeScript tidak menjamin semua bug hilang dan tidak otomatis memvalidasi JSON dari jaringan. Anggota tetap perlu mempelajari JavaScript, tipe, dan penanganan nilai kosong.

React: antarmuka dibagi menjadi komponen

React mendukung pemecahan UI menjadi komponen beserta pengelolaan state. [2] Ini cocok dengan kebutuhan project: kartu guru, formulir pendaftaran, daftar siswa, dan panel detail bisa dipisahkan serta dikembangkan per bagian.

Bukti lokal: komponen ada di `frontend/app/components/`; modul admin dipisahkan lagi di folder `admin/`, `school-directory/`, dan `student-records/`.

Keterbatasan: pemisahan komponen tidak otomatis membuat kode mudah dirawat. Tim tetap perlu menghindari komponen terlalu besar, data ganda, dan perubahan state yang sulit dilacak.

CSS dan Tailwind: tampilan konsisten, tetap fleksibel

Tailwind menyediakan utility class untuk styling. [3] Project ini juga menggunakan banyak CSS khusus yang dipisah berdasarkan area tampilan. Jadi penjelasannya bukan “seluruh desain dibuat Tailwind”, melainkan kombinasi CSS modular dan Tailwind.

Catatan penting: Vinext bukan Next.js standar

Vinext menyediakan kompatibilitas API bergaya Next.js di atas Vite. [4][5] Manifest lokal masih mendeklarasikan versi beta. Import seperti `next/link` tidak cukup menjadi alasan menyebut runtime project ini Next.js resmi.

Alasan mempertahankan sementara: struktur halaman dan komponen yang sudah ada dapat diteruskan untuk mockup lokal. **Risikonya:** kompatibilitas dan cara deployment perlu dievaluasi sebelum production. Konfigurasi saat ini memakai Cloudflare plugin dan Wrangler; jalur menjalankan frontend di VPS belum diselesaikan.

> “Kami memakai React dengan tooling Vinext/Vite untuk prototype. Untuk production di VPS, kami masih perlu memvalidasi runtime dan deployment-nya.”

BAGIAN 05

Mengapa Node.js dan Fastify?

Node.js: bahasa kerja frontend dan backend konsisten

Node.js adalah runtime JavaScript di luar browser. [6] Backend project ditulis dalam TypeScript dan memiliki proses build ke JavaScript. Konsistensi bahasa membantu tim memakai konsep data, fungsi, dan async yang sama pada dua sisi aplikasi.

Ini tidak berarti frontend boleh langsung mengakses database. Browser mengirim request ke API; backend memvalidasi, memeriksa izin, lalu mengakses database.

Fastify: dasar API yang dapat disusun per modul

Fastify menyediakan dukungan TypeScript dan validasi request berbasis schema. [7][8] Dalam project ini, plugin digunakan untuk cookie, CORS, header keamanan, pembatasan request, dan multipart. Modul autentikasi dipisahkan menjadi route, service, dan repository.

- **Kesesuaian:** kebutuhan utama backend adalah API konten dan administrasi pendaftar, bukan sistem layanan terdistribusi yang kompleks.
- **Pemeliharaan:** route mengurus HTTP, service mengurus aturan bisnis, repository mengurus query database.
- **Konsistensi:** pola modul bisa dijadikan contoh untuk anggota yang membuat fitur baru.

Keterbatasan: Fastify tidak otomatis menyediakan seluruh CMS, aturan hak akses, upload aman, atau halaman admin. Tim tetap harus mengimplementasikan dan menguji semuanya. Project ini juga belum memiliki pengukuran performa yang membuktikan pilihan paling cepat.

Kalau dosen bertanya: mengapa bukan Laravel atau Express?

“Laravel dan Express tetap bisa dipakai. Pertimbangan kami adalah kecocokan dengan kode yang sudah menggunakan TypeScript dan pola API modular. Kami tidak menyimpulkan Fastify selalu lebih baik. Kalau kemampuan tim dan kebutuhan operasional lebih sesuai dengan stack lain, itu layak dievaluasi sebelum scope membesar.”

Trade-off yang harus diakui: keseragaman bahasa membantu, tetapi tim pemula tetap perlu belajar HTTP, async, database, autentikasi, dan pengujian. Tidak ada framework yang menghilangkan kebutuhan belajar tersebut.

Mengapa tidak microservices?

Targetnya satu sekolah dengan satu backend modular. Memisahkan layanan per fitur belum menunjukkan kebutuhan yang jelas dan akan menambah koordinasi deployment. Pemisahan frontend/backend di folder berbeda tidak berarti sistem ini sudah memakai microservices.

Alasan utama yang kuat: kesesuaian kebutuhan, konsistensi bahasa, dan pembagian modul. Hindari jawaban “karena paling modern” atau “pasti paling cepat”.

BAGIAN 06

Mengapa PostgreSQL, Drizzle, dan OpenAPI?

PostgreSQL: data sekolah memiliki hubungan

PostgreSQL adalah database relasional dengan dukungan integritas data dan transaksi. [9] Kebutuhan project bersifat terstruktur: satu pendaftar dapat memiliki beberapa dokumen, satu admin dapat memiliki beberapa session, dan status pendaftaran harus memiliki nilai yang konsisten.

Schema awal yang tersedia: `admins`, `admin_sessions`, `applications`, dan `application_documents`. Relasinya mencakup admin ke session dan pendaftar ke dokumen. Nomor pendaftaran memiliki indeks unik. Namun schema dan migration yang tersedia belum membuktikan seluruh fitur menyimpan data ke database.

Mengapa bukan hanya browser storage? Data browser terikat pada browser/perangkat dan tidak menjadi pusat data bersama. Admin sekolah nantinya membutuhkan data yang sama walaupun membuka perangkat berbeda.

Mengapa bukan MySQL? MySQL juga mungkin memenuhi kebutuhan ini. PostgreSQL dipilih sebagai rancangan database relasional project, bukan karena MySQL dianggap tidak mampu. Tidak ada benchmark perbandingan database dalam project saat ini.

Drizzle: penghubung kode TypeScript dan SQL

Drizzle menyediakan ORM berorientasi TypeScript dengan pendekatan yang dekat dengan SQL. [10] Schema tabel didefinisikan di kode, query dibuat melalui ORM, dan Drizzle Kit membantu menghasilkan serta menjalankan migration.

Manfaat untuk tim: perubahan struktur tabel bisa ditinjau sebagai perubahan source dan SQL. **Batasnya:** anggota tetap perlu memahami relasi, indeks, transaksi, serta membaca migration sebelum dijalankan. ORM tidak otomatis membuat query efisien atau operasi hapus aman.

OpenAPI: kesepakatan frontend dan backend

OpenAPI mendeskripsikan antarmuka HTTP API. [11] File `contracts/openapi.yaml` menjadi acuan nama endpoint, field, respons, dan error. Misalnya anggota frontend perlu tahu data apa yang dikirim ketika pengguna mendaftar.

Penting: kontrak yang tertulis tidak membuat endpoint otomatis tersedia. Nama field dan aturan kontrak masih harus diselaraskan dengan implementasi setiap fitur. OpenAPI di sini adalah spesifikasi dokumentasi API, bukan layanan chatbot.

Zod dan validasi login berbeda fungsi

Zod adalah library validasi schema. [12] Saat ini penggunaannya terlihat untuk konfigurasi environment backend. Validasi body login menggunakan JSON Schema Fastify. Jangan menyebut seluruh form atau seluruh respons API sudah divalidasi Zod.

BAGIAN 07

Alur sistem dan struktur folder

Dua alur yang berbeda

Simulasi mayoritas fitur saat ini

```
Pengguna -> UI React -> local-cms -> storage browser
```

Konten publik menggunakan localStorage. Data pendaftar, pesan, dan newsletter menggunakan sessionStorage pada mockup. Data tersebut belum menjadi pusat data server bersama. Gunakan data fiktif; sessionStorage bukan mekanisme perlindungan data production.

Target setelah fitur diintegrasikan

```
Pengguna -> UI React -> API Fastify -> service -> repository -> PostgreSQL
```

Backend nantinya memeriksa format data, session dan izin jika diperlukan, menjalankan aturan bisnis, menyimpan data, lalu mengirim respons. UI harus menangani kondisi loading, sukses, kosong, dan gagal.

Pengecualian yang sudah disiapkan: UI login, pembacaan session, dan logout dapat memanggil API autentikasi jika alamat backend dikonfigurasi. Ini belum berarti semua menu admin menggunakan API.

Mulai membaca source dari mana?

Lokasi	Apa yang dijelaskan
`frontend/app/`	Route halaman, seperti beranda, blog, struktur, dan admin.
`frontend/app/components/`	Bagian UI yang dipisah menurut fungsi.
`frontend/app/lib/cms/`	Pengelolaan data mockup di browser.
`frontend/app/lib/api-client.ts`	Helper request ke backend; bukan seluruh integrasi fitur.
`backend/src/app.ts`	Registrasi plugin dan route API.
`backend/src/modules/auth/`	Contoh route, service, dan repository yang sudah ada.
`backend/src/db/schema/`	Definisi tabel database.
`backend/drizzle/`	Migration SQL yang perlu direview.
`contracts/openapi.yaml`	Acuan komunikasi frontend-backend.
`docs/README-TIM-PKM.md`	Rencana pembagian kerja 10 anggota.

Contoh penelusuran satu fitur

Mulai dari `frontend/app/pendaftaran/page.tsx`, lanjut ke `RegistrationForm.tsx`, lalu telusuri fungsi data di `lib/cms/`. Ketika API pendaftaran dibuat, alur penyimpanan ini perlu diganti secara terarah dan diuji. Jangan hanya menambahkan route backend tanpa mengubah pemanggilan frontend.

BAGIAN 08

Sudah dibuat atau belum?

Status berikut berdasarkan pemeriksaan kode lokal, bukan persentase completion dan bukan hasil uji seluruh alur bisnis.

Bagian	Kondisi saat ini	Pekerjaan berikutnya
Beranda, tentang, program	Halaman tersedia; konten contoh.	Validasi isi dengan sekolah; integrasikan data yang perlu dinamis.
Blog dan agenda	UI publik/admin dan pengelolaan mockup tersedia.	CRUD backend, aturan publikasi, dan koneksi UI.
Struktur dan guru	UI tambah/edit/hapus/urut tersedia secara lokal.	Penyimpanan server, validasi hierarki, izin admin.
Pendaftaran online	Form dan simulasi alur tersedia.	Endpoint submit, validasi, nomor unik, penyimpanan server.
Pendaftar offline dan pencarian	UI pencatatan, pencarian, dan detail tersedia.	Endpoint input/search/detail dan konsistensi sumber data.
Berkas siswa	UI/metadata mockup dan schema awal ada.	Upload, akses privat, download, verifikasi, dan uji izin.
Kontak, pesan, newsletter	Form dan data simulasi tersedia.	Persistensi server dan aturan pengelolaan; bukan layanan kirim email massal.
Dashboard/settings	Tampilan tersedia; sebagian data contoh/lokal.	Ringkasan dari database dan endpoint pengaturan.
Autentikasi admin	Kode login/session/logout, hash, dan cookie tersedia.	Uji terhadap database nyata dan perluasan otorisasi resource.
Database	Client, empat tabel, migration awal tersedia.	Validasi migration lokal dan tambah schema modul yang belum ada.
Infrastruktur	Dockerfile dan Compose tersedia.	Uji container dan finalisasi runtime frontend serta konfigurasi VPS.

Kalimat progres yang aman

“Frontend sudah menjadi prototype interaktif. Backend baru fondasi autentikasi, health check, konfigurasi, dan schema awal. Integrasi fitur sekolah masih dikerjakan per modul.”

Bukti yang jangan dianggap setara

- Ada tabel bukan berarti API CRUD sudah ada.
- Tombol berhasil di demo bukan berarti data masuk PostgreSQL.
- Ada pilihan role bukan berarti seluruh endpoint sudah memeriksa role.
- Ada file Docker bukan berarti aplikasi telah berhasil dideploy.
- Daftar route dalam OpenAPI bukan daftar endpoint yang semuanya sudah hidup.

BAGIAN 09

Keamanan: fondasi dan pekerjaan wajib

Yang terlihat dalam kode saat ini

Password admin diproses dengan Argon2. Session memakai token acak; hash token disimpan di database. Cookie diberi HttpOnly dan SameSite=Lax, serta Secure ketika environment production. Backend memiliki CORS allowlist, Helmet, rate limit, dan batas multipart.

Arti sederhananya: password tidak disimpan sebagai teks biasa; session memiliki masa berlaku; akses lintas origin browser dibatasi; percobaan request dibatasi. Itu lapisan dasar, bukan sertifikat bahwa aplikasi sudah aman.

Yang belum boleh diklaim selesai

- Otorisasi setiap modul: siapa boleh melihat identitas, mengubah status, atau mengambil dokumen siswa.
- Upload aman: pemeriksaan jenis/ukuran/isi file, nama simpan yang aman, dan penyimpanan nonpublik. [13]
- Perlindungan request yang mengubah data terhadap CSRF; kebijakan session dan cookie perlu diuji sesuai domain deployment. [14]
- Audit aktivitas admin, kebijakan retensi data, backup/restore, dan penanganan insiden.
- Pengujian login database, akses tanpa login, akses lintas role, dan kebocoran informasi pada error/log.

Mengapa berkas tidak ditaruh di folder public?

KK, akta, ijazah, dan identitas siswa tidak boleh dapat diambil hanya dengan mengetahui URL. Rancangan project menyimpan metadata di database, sedangkan file ditempatkan di penyimpanan privat. Endpoint download harus memeriksa izin pengguna. [13] Endpoint tersebut belum tersedia sebagai fitur lengkap.

Pertanyaan yang sering menjebak

“Apakah semua data sudah terenkripsi?” Belum ada bukti implementasi enkripsi seluruh data tersimpan. Hash password bukan enkripsi seluruh database. Konfigurasi HTTPS, penyimpanan, dan backup harus diverifikasi terpisah.

“CORS berarti API tidak bisa diakses orang lain?” Tidak. CORS adalah kebijakan akses lintas origin di browser, bukan pengganti autentikasi atau otorisasi backend.

“SessionStorage sudah aman untuk data asli?” Tidak cukup untuk kebutuhan production. Gunakan data dummy dalam demonstrasi. Persetujuan pengumpulan data, minimisasi field, dan aturan akses harus dibahas dengan pihak sekolah.

“Keamanan dasar telah disiapkan, tetapi pengujian izin akses dan perlindungan dokumen masih harus diselesaikan sebelum menerima data siswa nyata.”

BAGIAN 10

Mengapa Agile, dan bukti penerapannya?

Agile menekankan hasil yang dapat digunakan, kolaborasi, adaptasi kebutuhan, dan evaluasi berkala. [15] Untuk project ini, pendekatan iteratif masuk akal karena alur admin dan kebutuhan dokumen perlu dikonfirmasi melalui demo dan masukan sekolah.

Batas klaim: source code tidak membuktikan Agile telah dijalankan. Jadwal di bawah adalah usulan, bukan catatan sprint yang sudah berlangsung. Agile juga bukan kewajiban memakai React, Fastify, atau framework tertentu.

Contoh iterasi 1-2 minggu, disesuaikan kalender tim

Tahap	Fokus hasil	Bukti yang disimpan
Persiapan	Konfirmasi masalah, pengguna, field, dan prioritas.	Notulen, kebutuhan, backlog awal.
Iterasi 1	Autentikasi dan satu alur pendaftaran tersimpan.	Task, perubahan kode, hasil uji, demo.
Iterasi 2	Pencarian/detail siswa, input offline, dokumen privat.	Skenario uji izin dan feedback admin.
Iterasi 3	Konten, direktori guru, dan settings terintegrasi.	Review sekolah dan revisi backlog.
Iterasi 4	Perbaikan UI, UAT, backup, serta staging.	Hasil UAT, bug list, panduan operasional.

Setiap iterasi: pilih pekerjaan realistis, kerjakan, review kode, jalankan pengujian, demonstrasikan hasil, catat masukan, lalu perbaiki prioritas. Jangan menunggu semua menu selesai baru mengintegrasikan API.

Contoh user story dan penerimaan

User story: “Sebagai admin pendaftaran, saya ingin mencari calon siswa berdasarkan nama atau nomor pendaftaran agar dapat membuka data dan kelengkapan berkasnya.”

Kriteria selesai: hasil sesuai pencarian; kondisi kosong jelas; admin berizin dapat membuka detail; pengguna tanpa izin ditolak backend; data tetap ada setelah browser dibuka kembali karena berasal dari database; skenario berhasil dan gagal tercatat.

Agile bukan metode pengujian

Agile menjelaskan cara mengelola pengerjaan. Black-box testing memeriksa perilaku input/output; UAT menilai penerimaan pengguna terhadap kebutuhan. Jika tim ingin menyebut Scrum secara khusus, peran, aktivitas, dan artefaknya perlu benar-benar diterapkan, bukan hanya menggunakan istilah sprint.

Nilai PKM: penerapan yang sesuai kebutuhan sekolah, pendampingan pengguna, serta evaluasi hasil. Jangan mengklaim inovasi algoritma baru atau peningkatan efisiensi sebelum ada pengukuran.

BAGIAN 11

Pengujian dan rencana deploy ke VPS

Pengujian yang tersedia di source

Frontend memiliki 5 test rendered HTML dan 7 test utilitas CMS. Backend memiliki 3 test: health response, route tidak ditemukan, dan payload login tidak valid. Pemeriksaan root mencakup lint, typecheck, dan test; test frontend juga menjalankan build.

Batas penting: test backend tersebut tidak membuktikan login berhasil ke PostgreSQL. Test HTML juga tidak membuktikan seluruh interaksi mobile atau alur pendaftaran production berhasil. Pada penyusunan dokumen ini test tidak dijalankan ulang; jumlah dan lingkup diperiksa dari file test.

Pengujian lanjutan yang perlu disiapkan

Area	Contoh bukti yang perlu dikumpulkan
Fungsional	Pendaftaran valid tersimpan, input salah ditolak, nomor unik.
Integrasi	UI mengirim field yang benar; API menyimpan; detail dibaca kembali.
Keamanan	Tanpa login ditolak; role tidak berizin gagal membuka dokumen.
UI/UX	Form, tabel, navigasi, fokus keyboard, dan pesan error di HP/desktop.
UAT	Admin sekolah menjalankan tugas nyata dengan data uji lalu memberi masukan.
Operasional	Migration diuji; backup dapat dipulihkan; restart layanan tidak menghilangkan data.

Jika mengukur waktu pencarian atau kepuasan pengguna, tentukan skenario, peserta, perangkat, dan cara menghitung hasil. Jangan menulis “lebih cepat 80%” atau “100% aman” tanpa data yang mendukung.

Target VPS satu sekolah

Rancangan: browser mengakses HTTPS, reverse proxy meneruskan request ke frontend/API, lalu backend mengakses PostgreSQL melalui jaringan privat. Upload privat dan backup disiapkan terpisah dari file publik. Ini rancangan deployment, belum kondisi server yang diverifikasi.

Docker membantu mengemas lingkungan aplikasi; Compose mendeskripsikan beberapa layanan. [16] Project memiliki template backend/database, tetapi container dan deployment penuh belum dibuktikan dalam dokumen ini. `wrangler dev` pada script frontend adalah preview pengembangan, bukan prosedur production VPS final.

Keputusan sebelum deploy: runtime frontend, domain dan rute API, HTTPS, akses admin, kapasitas yang diuji, backup dan restore, serta siapa yang merawat sistem. Tidak ada angka biaya atau kapasitas VPS yang dijanjikan tanpa pengukuran.

BAGIAN 12

Latihan tanya jawab: konsep dan pilihan

1. Bahasa pemrogramannya apa?

“Utamanya TypeScript, yaitu JavaScript dengan pemeriksaan tipe. Komponen React menggunakan file .tsx, logika menggunakan .ts. SQL digunakan untuk kebutuhan database dan migration.”

2. Framework-nya apa?

“Backend menggunakan Fastify. Frontend menggunakan library React dengan Vinext/Vite sebagai tooling routing, rendering, dan build. Styling memakai CSS dan Tailwind.”

3. Mengapa TypeScript, bukan JavaScript biasa?

“Bentuk data lebih eksplisit ketika fitur dikerjakan beberapa anggota. Kesalahan tipe tertentu dapat diketahui sebelum program berjalan. Kami tetap membutuhkan validasi runtime dan testing karena TypeScript tidak memeriksa semua kesalahan.”

4. Mengapa Fastify?

“Cocok dengan backend TypeScript, memiliki dukungan schema dan plugin, serta dapat dibagi per modul. Alasan kami kesesuaian dan kemudahan pengaturan kode, bukan hasil klaim bahwa framework ini pasti paling cepat.”

5. Mengapa frontend dan backend dipisah?

“Agar UI, aturan bisnis, dan akses database memiliki tanggung jawab berbeda. Tim dapat mengerjakan modul dengan acuan kontrak API. Konsekuensinya kami perlu menyelaraskan field, respons error, autentikasi, dan deployment kedua sisi.”

6. Mengapa memakai PostgreSQL?

“Data pendaftar, dokumen, dan admin memiliki relasi yang jelas. Database relasional sesuai untuk menjaga hubungan dan konsistensi tersebut. Alternatif relasional lain juga memungkinkan; pilihan ini mengikuti rancangan project.”

7. OpenAPI itu untuk apa?

“Sebagai kesepakatan format komunikasi frontend dan backend. File kontrak menjelaskan endpoint dan data yang diharapkan. Implementasinya tetap perlu dibuat dan diuji; dokumentasi saja tidak menjalankan endpoint.”

8. Mengapa berbasis web, bukan aplikasi Android?

“Ruang lingkup kami informasi publik dan administrasi melalui browser, sehingga tidak memerlukan instalasi aplikasi khusus. Mobile tetap menjadi target tampilan. Jika nantinya ada kebutuhan fitur perangkat atau offline khusus, itu perlu evaluasi terpisah.”

BAGIAN 13

Latihan tanya jawab: progres dan keterbatasan

9. Apakah project sudah selesai?

“Belum. UI sudah menjadi simulasi interaktif dan fondasi backend tersedia. API resource serta integrasi fitur ke database masih perlu diselesaikan. Kami menunjukkan bagian yang berfungsi sebagai demo dan menandai yang belum terintegrasi.”

10. Kalau daftar sekarang, datanya masuk ke mana?

“Pada mode mockup, data pendaftaran masih menggunakan sessionStorage browser. Target akhirnya PostgreSQL melalui API, tetapi alur tersebut belum selesai untuk fitur pendaftaran.”

11. Apakah login berarti semua data sudah dari server?

“Tidak. UI autentikasi dapat memakai backend jika dikonfigurasi, tetapi menu lain masih banyak yang menggunakan adapter mockup. Login dan integrasi CRUD adalah pekerjaan berbeda.”

12. Bagaimana memastikan aman untuk dokumen siswa?

“Kami merencanakan penyimpanan privat, validasi file, dan pemeriksaan izin pada download. Fondasi session sudah ada, tetapi pengujian upload dan otorisasi dokumen belum selesai. Karena itu demo memakai data dummy.”

13. Bukti metode Agile-nya apa?

“Buktinya harus berupa backlog, pekerjaan per iterasi, hasil demo, feedback sekolah, dan perbaikan berikutnya. Kami tidak menjadikan keberadaan framework sebagai bukti Agile. Jadwal dan catatan aktual akan kami dokumentasikan.”

14. Apa ukuran keberhasilan project?

“Kebutuhan prioritas disepakati sekolah, alur pendaftaran dan pencarian data lulus uji, pengguna berizin dapat mengelola data, serta admin sekolah menerima hasil melalui UAT. Target pengukuran akan disepakati sebelum evaluasi, bukan dibuat setelah hasil diketahui.”

15. Siapa yang merawat setelah tim selesai?

“Perlu ditetapkan operator sekolah dan penanggung jawab teknis. Serah terima mencakup akun melalui cara aman, panduan penggunaan, backup/restore, deployment, serta prosedur pembaruan. Belum cukup hanya menyerahkan ZIP source.”

16. Kalau ditanya hal yang belum dipahami?

“Bagian itu belum saya verifikasi. Yang sudah dapat saya tunjukkan adalah [file atau alur yang dipahami]. Saya akan memeriksa implementasi dan pengujiannya sebelum menyimpulkan.” Jawaban ini lebih tepat daripada mengarang hasil atau kemampuan sistem.

BAGIAN 14

Persiapan demo dan langkah berikutnya

Urutan demo sekitar 5 menit

- **Menit 1:** jelaskan masalah, pengguna, dan ruang lingkup; tandai bahwa demo masih lokal.
- **Menit 2:** tunjukkan beranda, blog, dan struktur/guru. Jelaskan kontennya contoh, bukan profil resmi yang sudah disahkan sekolah.
- **Menit 3:** coba pendaftaran dengan data dummy. Sebutkan bahwa penyimpanannya masih browser pada mode mockup.
- **Menit 4:** buka admin, cari pendaftar, dan tampilkan detail. Bedakan contoh berkas dari layanan upload privat yang belum selesai.
- **Menit 5:** tunjukkan pemisahan folder, contoh modul auth, daftar pekerjaan berikutnya, lalu minta feedback kebutuhan prioritas.

Perintah lokal untuk demo frontend

Jalankan `npm ci`, kemudian `npm run dev` dari folder project yang memiliki `package.json`. Buka `http://localhost:3000`; dashboard di `/admin`. Untuk mode demo, ikuti `PANDUAN-INSTALASI.md`. Jangan menampilkan file environment atau credential saat presentasi.

Urutan pengerjaan yang disarankan

1. Konfirmasi kebutuhan sekolah dan definisi selesai setiap fitur.
2. Jalankan serta uji database dan autentikasi terhadap database nyata.
3. Selesaikan satu alur vertikal: form pendaftaran -> API -> database -> daftar/detail admin.
4. Tambahkan input offline, pencarian, aturan status, dan dokumen privat.
5. Integrasikan konten, guru/struktur, pesan, settings, dan ringkasan dashboard.
6. Jalankan uji fungsional, keamanan, UI/UX, UAT, serta backup/restore sebelum production.

Tim 10 orang: bagaimana menjelaskannya?

Pembagian mengikuti fitur, bukan sepuluh orang menyunting file yang sama. Rencana yang sudah ada mencakup: (1) beranda/tentang dan UI settings; (2) program/agenda; (3) struktur/guru; (4) blog; (5) pendaftaran online; (6) data siswa/offline dan UI dokumen; (7) backend dokumen; (8) kontak/pesan dan koordinasi QA; (9) autentikasi dan backend settings; (10) ringkasan dashboard dan koordinasi integrasi.

Ini pembagian rencana, bukan bukti kontribusi. Sesuaikan kemampuan anggota dan gunakan pendampingan untuk autentikasi/dokumen. Pemilik fitur bertanggung jawab memperbaiki bug fiturnya; koordinator bukan penanggung jawab seluruh pekerjaan yang tertunda. Path dan checklist lengkap ada di `docs/README-TIM-PKM.md`.

Bawa saat bimbingan: demo data dummy, daftar progres jujur, backlog prioritas, diagram/alur sederhana, dan pertanyaan untuk sekolah/dosen. Pahami satu alur dari UI sampai penyimpanan sebelum mencoba menghafal seluruh dependency.

BAGIAN 15

Referensi dan dasar pemeriksaan

Dokumentasi resmi

Referensi diakses 28 September 2026. Sumber menjelaskan kemampuan teknologi; alasan kesesuaiannya dengan project merupakan pertimbangan desain, bukan kesimpulan benchmark. Nomor referensi dirujuk di bagian terkait.

- [1] TypeScript Handbook
- [2] React: Thinking in React
- [3] Tailwind CSS: Styling with utility classes
- [4] Vinext: repositori dan dokumentasi resmi
- [5] Vite: Getting Started
- [6] Node.js: Introduction
- [7] Fastify: TypeScript
- [8] Fastify: Validation and Serialization
- [9] PostgreSQL: About
- [10] Drizzle ORM: Overview
- [11] OpenAPI Specification
- [12] Zod: Introduction
- [13] OWASP: File Upload Cheat Sheet
- [14] OWASP: Session Management Cheat Sheet
- [15] Prinsip-prinsip Manifesto Agile
- [16] Docker: Overview

Bukti lokal yang diperiksa

Manifest root/frontend/backend; `frontend/vite.config.ts`; `frontend/app/globals.css`; `frontend/app/lib/cms/storage.ts`; `frontend/app/lib/api-client.ts`; komponen AdminLogin dan AdminDashboard; `backend/src/app.ts`; modul auth/health; schema auth/admissions; konfigurasi environment; file test frontend/backend; README backend; kontrak API dan panduan tim.

Pemeriksaan untuk dokumen ini bersifat baca source dan dokumentasi. Tidak menjalankan migration, tidak mengisi data siswa asli, tidak melakukan deployment, dan tidak mengubah fitur aplikasi.

Perbarui sebelum bimbingan berikutnya: status integrasi fitur, hasil test aktual, keputusan runtime VPS, temuan wawancara sekolah, serta bukti iterasi dan kontribusi anggota. Dokumen ini adalah potret kondisi tanggal pemeriksaan, bukan jaminan kondisi project selamanya.